

Releasing Is What Matters: Lessons from Coding and Music

In both software development and music production, one truth stands out: **releasing your work is what truly counts**. There is a common myth among creators that success comes from crafting the perfect code or the perfect song in isolation. Musicians may spend years polishing an album, and programmers might obsess over writing flawless code. But whether you're an indie artist or a software developer, simply having a great product hidden on your hard drive isn't enough. **The real value emerges only when you put your work out into the world**. In today's fast-paced digital era, releasing early and often can make all the difference in connecting with an audience and improving your craft.

The Myth of the "Perfect Debut"

Many creators fall into the trap of the **"perfect debut" myth** – the idea that you get only one big chance to impress the world. In music, this often means stockpiling songs until that *one* immaculate album or single can be released. In coding, it manifests as developers holding back a program until every feature is polished and every bug squashed. The myth suggests that if the first release isn't flawless, your audience will never give you a second chance. **This mindset is misguided**. If a song or app fails to resonate, odds are most people won't even notice its release – and if it *does* resonate, people are usually willing to forgive early imperfections. As LinkedIn co-founder Reid Hoffman famously put it, *"If you're not embarrassed by the first version of your product, you've launched too late."* ¹. In other words, waiting forever for perfection often means missed opportunities and **launching too late**.

Perfectionism can actually hinder progress. A musician who never shares their music gains no listeners, and a programmer who never ships code gains no users. What's the point of a masterpiece no one ever hears or an ingenious program no one can run? **The pursuit of perfection can become procrastination**. Meanwhile, creators who consistently release their work tend to improve faster and build an audience over time. The truth is that you **learn more from putting something out there** than from endlessly refining in private. It's only after release that you get **real feedback** and see how your creation connects with people.

Connection Over Perfection

Art and software only fulfill their purpose when they connect with a user or listener. In music, the ultimate goal isn't a perfectly engineered track – it's making listeners *feel* something. A lo-fi song that moves people will have far more impact than a sonically pristine track that leaves listeners cold. The **emotional response and connection** are what count. As one music industry blog notes, *"releasing music consistently keeps your name in circulation, deepens fan relationships, and builds a lasting presence"* ². Listeners care about songs that speak to them or energize them at the right moment, not the bit rate of the audio or the pedigree of the studio equipment. Some iconic songs were recorded in basements or had technical flaws, but fans love them because of the human feeling they convey.

The same principle applies to coding. **Code is a means to an end**; it's the tool through which we deliver value or solve a problem for users. Users don't rave about how elegant your code is behind the scenes – they care that your app makes their life easier or more enjoyable. A simple app that addresses a real

pain point will beat a beautifully architected system that doesn't actually solve a pressing need. In software startups, this is the essence of *product-market fit*: does your software solve a problem people actually have, and do they find it useful enough to adopt or even pay for? A program could be written in messy code, but if it **connects with users and provides value**, it has a far greater chance of success than a perfectly coded project that nobody wants. In short, **what matters is the connection and value delivered to the end user, not the perfection of form**.

It's not that quality, engineering, or production values are unimportant – they do matter, especially as you scale up. But obsessing over quality before you have something that people want is putting the cart before the horse. A slickly produced album means nothing if no one plays it, and a well-engineered software system is wasted if no one needs it. **First, make something people want; then you can work on making it great.**

“Real Artists Ship” – Release Early and Iterate

Steve Jobs famously said, *“Real artists ship.”* This Silicon Valley mantra underlines that **creating value requires shipping your work to the world** ³. In software, code that isn't released might as well not exist – it provides no value sitting in a private repository. *“Software only has value when it actually ships,”* as one engineer put it; unshipped code is just potential energy waiting to be realized ⁴. Likewise in music, a song that stays on your laptop might be the greatest track never heard. **Releasing turns potential into actual impact.** It's the crucial step where an idea meets reality.

Releasing early (even if the product or song is rough) has several benefits. **First, you learn by doing.** A software team often finds that the last 10% of getting a product to production teaches them more than the first 90% of coding ⁵. Pushing code out to real users reveals unexpected bugs, edge cases, and performance issues that you simply can't anticipate in isolation. *“Doing the work to push things through to production inherently de-risks things, because we learn what the challenges are... And when software goes out to users, we get to hear their feedback as well,”* one engineering blog notes ⁵. The same goes for music: by publishing a song, an artist learns how listeners react. Which tracks do people replay or skip? Do they share it with friends? This feedback is invaluable, and **you can only get it by releasing the work.**

Second, frequent releases build momentum and skill. In music, many successful independent artists have adopted a strategy of steady output – releasing singles or EPs every few weeks or months rather than waiting years between albums. This keeps fans engaged and allows the artist to practice the craft of releasing itself. Each release is an occasion to promote, get listener feedback, and refine marketing strategies. In the streaming era, consistent output can even trigger algorithmic advantages on platforms like Spotify (which tend to favor regular activity). Releasing music regularly *“ensures your name stays in playlists, conversations, and search results,”* helping you stay relevant ⁶. And from a creative standpoint, **the more you create and share, the more you hone your skills** ⁷. You learn what resonates with your audience and improve your songwriting or production with each iteration.

In software, the analog is the agile/lean approach: ship small increments often. By delivering updates or MVPs early and frequently, you practice the whole cycle of development, release, feedback, and iteration. You'll become better at deployment, at gathering user input, and at adjusting your product quickly. Releasing is a skill, and **the only way to master it is to keep doing it.** Moreover, users appreciate continuous improvements. An app that steadily gets better (even if it started basic) can earn more loyalty than one that appears “perfect” at launch but then stagnates. Frequent releases signal to users or customers that you are actively improving the product and listening to their needs.

From a strategic perspective, when you're working on something novel – what strategy expert Simon Wardley would call the *Genesis* or *Custom-Built* stage of a product – **rapid iteration and exploration are key**. Early on, you don't fully know what the "right" solution or feature set is; the best way to find out is to **get a working version in front of people and see what happens**. This is essentially the philosophy of the Lean Startup: release a *Minimum Viable Product* to start the learning process. **You can't accurately predict what users will love or hate without letting them experience it**. By treating each release as an experiment, you gather data to guide your next steps.

Feedback: The Unsung Hero of Success

One major reason to release early and often is to **get feedback**. Before release, everything is just an assumption – you *think* people will like your new song's chorus, or you *assume* users need the feature you coded. After release, you have actual data. Maybe listeners overwhelmingly love the song's energy but comment that the intro is too long. Maybe users of your app gravitate to one feature and ignore another, revealing what they truly value. **This feedback is gold**, and it often **challenges your initial assumptions**, helping you course-correct.

Crucially, releasing turns creation into a two-way conversation. Instead of guessing in a vacuum, you're interacting with an audience. This is how you **find product-market fit or artistic resonance**: by adjusting based on real responses. It's worth noting that **feedback is not just about external responses, but also internal learning**. Completing a project and seeing it out in the wild gives you a better understanding of the full process, from idea to execution to user experience. Each release makes you a stronger creator because you've seen what happens after the "launch" button is pressed.

Additionally, feedback loops help ensure that *quality improves over time*. Early releases might be raw, but thanks to user feedback, each subsequent version can get closer to what the audience wants. A musician might discover that their audience really connects with songs that have personal lyrics, prompting them to focus more on that in future compositions. A developer might learn that their software is being used in a way they didn't expect, leading them to tweak the interface or add a crucial feature. **None of this insight would come without putting an imperfect version out there and listening**.

Overcoming the Fear of Release

It's natural to feel anxious about releasing your work. Creators often wrestle with thoughts like: "*What if people hate it? What if it's not good enough yet? What if someone steals my idea once it's out there?*" These fears can be paralyzing, but they can be overcome by reframing what release really means.

First, forget the notion that you only get one shot. In reality, if your first release doesn't make a splash, very few people will notice — which means you *do* get another shot, and another after that. Your early audience is likely small and forgiving; they're early adopters who are excited to be part of a new thing. Over time, as you improve, those who jumped on early will feel proud they "discovered" you, and new audience members will mostly pay attention to your latest and greatest work, not the rough-cut demo you put out a year ago. **You are allowed to grow in public**. Think of each release as chapter one, chapter two, and so on of an ongoing story, rather than a once-in-a-lifetime debut on the world stage.

Second, consider the cost of not releasing. A song not released will definitely earn zero fans. A program not launched will solve zero problems. By holding back, you guarantee failure (in terms of impact), whereas by releasing, you at least give yourself a chance to succeed. Most creators regret the things they *didn't* share more than the things they did. Remember that **an imperfect release that**

sparks some interest beats a perfect project that lives only in your head. And if your work isn't well-received initially, that feedback is exactly what you need to make it better next time. As we noted earlier, even LinkedIn's founder encourages embracing a bit of embarrassment in early versions ¹ – it's a sign you're learning and improving.

What about someone copying your idea? This is a common worry, especially for startups and entrepreneurs. The hard truth is, if an idea is truly valuable, competition will emerge sooner or later. If you release and your idea has merit, yes, others might imitate it – but by then you have the **first-mover advantage** and a growing user base. You'll also be learning from users and moving forward, which is something a copycat will always lag behind on. On the other hand, if you never release, someone else might independently develop a similar idea and actually execute it – and they'll reap the rewards while you're left holding an unrealized vision. Generally, **execution and momentum matter more than the idea alone.** By the time competitors catch up, you could be onto the next improvement. Also, building a brand and relationship with your audience can create loyalty that a mere copy can't easily steal. In music, fans often stick with an artist because of the personal connection and authenticity, not just a single song. In software, users might prefer your app because of better support, community, or constant updates. Those are advantages you gain by actually being in the arena, releasing and engaging, not by hiding an idea out of fear.

Finally, realize that **releasing is not a spectator sport.** You can't learn how to swim by reading about it on the sidelines; at some point you have to jump in the water. Similarly, you can't fully prepare for everything that comes with public releases through theory alone. The anxiety before releasing something for the first time – that knot in your stomach – can only be untangled by the experience of *actually doing it*. Once you ship a few projects or drop a few tracks, you'll notice the fear subsides. You start focusing more on the *process* and *iteration* and less on a single moment of judgment. By making releasing a routine rather than a rare, high-stakes event, you desensitize those fears. Each release becomes just another step forward, not a make-or-break apocalypse in your mind.

Bringing Users and Listeners Along for the Journey

One often overlooked benefit of early releases is that you can **bring your audience along for the journey.** Early adopters — whether they are beta users of an app or the first fans of an indie band — love being part of the growth process. When you share something a bit raw and then continuously improve it, your earliest supporters feel a sense of *investment and pride*. They were there when it was rough around the edges; they provided feedback that helped shape the next version; they watched it evolve. This creates a deeper bond between creator and audience.

In the music world, fans sometimes cherish an artist's demo tapes or early recordings because they showcase authenticity and evolution. An audience that's involved from the beginning will often be more **loyal and enthusiastic**, because they've had a hand (even indirectly) in molding the final product. They'll happily spread the word: *"I remember when this app was just a simple tool with one feature, and now look at it!"* or *"I've been following this singer since her first rough EP – it's amazing how far she's come, and I feel like I was part of that."* Early adopters often prefer the more *raw* versions; it makes them feel special, like they've discovered something before the rest of the world. By sharing your journey, you turn users and listeners into **community members**, not just consumers.

There is also a personal creative benefit to this approach: it keeps you honest and motivated. If you know you have a small audience waiting for your next release or update, you're driven to keep creating. The journey mindset shifts your focus from fearing the audience to embracing them as collaborators in your improvement. In both coding and music, the creators who succeed in the long run often treat their

audience not as distant judges but as partners. **Releasing early and iterating openly signals humility and a willingness to improve**, which many people appreciate. It shows that you care about making something *for them*, not just about your own perfectionism.

Balancing Quality with Value

It's worth addressing a subtle point: advocating for early releases is not the same as advocating for sloppy work or ignoring quality entirely. **Quality does matter** – security, usability, and reliability are important in software; and things like melody, rhythm, and mix do matter in music. The key is **sequencing and balance**. At the start, value and connection trump polish. You focus on the core that delivers value: the feature that solves a problem, or the melody that carries the song's emotion. Once that core is validated by user/listener response, you invest in **refinement and scale**.

In software development, this might mean you release a basic version of an app that people find useful, and only after seeing real adoption do you spend time refactoring code, optimizing performance, or scaling the architecture. It's a known pitfall to prematurely optimize or over-engineer something that hasn't proven its worth yet. By contrast, if you have **evidence that users need what you're building (they're signing up, they're using it despite rough edges)**, then you can justify hardening the code and improving the design. In fact, at that point, you must improve quality to keep users happy as you grow – technical debt eventually needs addressing. The difference is you'll be doing it with confidence that the effort is worthwhile, because you have users who will benefit from those improvements.

In music, balancing quality with value might look like this: you release a series of singles recorded in your home studio to gauge listener interest. When one of the songs starts getting significant traction – say, it connects strongly with an audience – you might decide to invest in a professional re-recording or hire an audio engineer to polish it further. Or perhaps you notice that fans love your songwriting but comment on the production quality, so you work on that aspect for the next release. The point is that **initially you focus on the song's appeal (value)**, and once that's validated, you **elevate the production quality** in areas that amplify that value. By iterating like this, you eventually reach a product that is both high quality *and* high value to listeners.

It's a delicate balance: you don't want to release complete garbage that could alienate users (**a basic level of functionality or listenability is important**), but you also shouldn't hold back a good idea for years just to polish every pixel or every note. The art is knowing what's "good enough" to test the waters. Remember, **version 1.0 is not the endgame; it's the starting line**. You can always improve on something that exists; you can't improve on something that was never released.

Conclusion: Shipping Connects Vision to Reality

Whether you're writing code or composing music, **releasing is the bridge between your vision and the audience's reality**. Until you ship that code or drop that track, your work remains a private fantasy. Once you release it, it gains the potential to live in other people's minds, to solve their problems, to brighten their day, or to inspire them. The act of release transforms a personal creation into a shared experience.

We've seen that in both domains, **code and music**, the act of releasing trumps the illusion of perfection. Great software and great songs are not born fully formed in isolation; they evolve through interaction with users and listeners. By focusing on connecting with your audience and delivering value early, you open the door to feedback, learning, and continuous improvement. You also overcome the inertia and fear that stop so many would-be creators from ever being heard or used at all.

In today's world, barriers to publishing code or music are lower than ever – app stores, open-source platforms, streaming services, and social media give you direct access to a global audience. This means the new bottleneck is often **mindset**, not distribution. The creators who thrive are the ones who consistently put work out there, listen to the response, and keep going. They understand that **the journey with an audience is more powerful than a solitary quest for perfection**.

So if you're a developer agonizing over whether your app is "ready," or a musician wondering if your song is "good enough," take this to heart: **get it out into the world**. Shipping that first version or releasing that rough-cut song is not the end – it's the beginning of a dialogue. Your code and your music are just the starting points for something bigger: a connection with the people who use what you've made. And that connection, once established, can be nurtured into something truly remarkable. After all, *real artists ship*, and the world needs to see (and hear) what you've been working on. Go ahead and release – the sooner you do, the sooner the real journey begins.

Sources:

- Hoffman, R. – *If you're not embarrassed by the first version of your product, you've launched too late!* ¹
- Jobs, S. – *"Real artists ship."* ³ (implying that creative work only matters when delivered)
- McCormick, B. – *How Software Engineers Produce Value* (on the importance of shipping code and learning from it) ⁵
- ArtistRack Blog – *The Power of Consistency: Releasing Music Regularly* (benefits of frequent music releases for engagement) ²

¹ If You're not Embarrassed by the First Version of your Product you've Launched too Late | jackiestapleton.com

<https://jackiestapleton.com/2023/11/09/if-youre-not-embarrassed-by-the-first-version-of-your-product-youve-launched-too-late/>

² ⁶ ⁷ The Power of Consistency: Why Releasing Music Regularly Is Key to Your Success as an Independent Artist - ArtistRack | #1 Music Promotion & Playlisting Service for Artists

<https://artistrack.com/the-power-of-consistency-why-releasing-music-regularly-is-key-to-your-success-as-an-independent-artist/>

³ ⁴ ⁵ How Software Engineers Produce Value - Herding Lions

<https://benmccormick.org/2019/12/17/190000.html>